

CHAPTER 9

EXCEPTION AND INTERRUPT HANDLING

At the heart of an embedded system lie the exception handlers. They are responsible for handling errors, interrupts, and other events generated by the external system. Efficient handlers can dramatically improve system performance. The process of determining a good handling method can be complicated, challenging, and fun.

In this chapter we will cover the theory and practice of handling exceptions, and specifically the handling of interrupts on the ARM processor. The ARM processor has seven exceptions that can halt the normal sequential execution of instructions: Data Abort, Fast Interrupt Request, Interrupt Request, Prefetch Abort, Software Interrupt, Reset, and Undefined Instruction.

This chapter is divided into three main sections:

- *Exception handling.* Exception handling covers the specific details of how the ARM processor handles exceptions.
- *Interrupts.* ARM defines an interrupt as a special type of exception. This section discusses the use of interrupt requests, as well as introducing some of the common terms, features, and mechanisms surrounding interrupt handling.
- *Interrupt handling schemes.* The final section provides a set of interrupt handling methods. Included with each method is an example implementation.

9.1 EXCEPTION HANDLING

An exception is any condition that needs to halt the normal sequential execution of instructions. Examples are when the ARM core is reset, when an instruction fetch or memory access fails, when an undefined instruction is encountered, when a software interrupt instruction is executed, or when an external interrupt has been raised. Exception handling is the method of processing these exceptions.

Most exceptions have an associated software *exception handler*—a software routine that executes when an exception occurs. For instance, a Data Abort exception will have a Data Abort handler. The handler first determines the cause of the exception and then services the exception. Servicing takes place either within the handler or by branching to a specific service routine. The Reset exception is a special case since it is used to initialize an embedded system.

This section covers the following exception handling topics:

- ARM processor mode and exceptions
- Vector table
- Exception priorities
- Link register offsets

9.1.1 ARM PROCESSOR EXCEPTIONS AND MODES

Table 9.1 lists the ARM processor exceptions. Each exception causes the core to enter a specific mode. In addition, any of the ARM processor modes can be entered manually by changing the *cpsr*. *User* and *system* mode are the only two modes that are not entered by a corresponding exception, in other words, to enter these modes you must modify the *cpsr*.

When an exception causes a mode change, the core automatically

- saves the *cpsr* to the *spsr* of the exception mode
- saves the *pc* to the *lr* of the exception mode

Table 9.1 ARM processor exceptions and associated modes.

Exception	Mode	Main purpose
Fast Interrupt Request	<i>FIQ</i>	fast interrupt request handling
Interrupt Request	<i>IRQ</i>	interrupt request handling
SWI and Reset	<i>SVC</i>	protected mode for operating systems
Prefetch Abort and Data Abort	<i>abort</i>	virtual memory and/or memory protection handling
Undefined Instruction	<i>undefined</i>	software emulation of hardware coprocessors

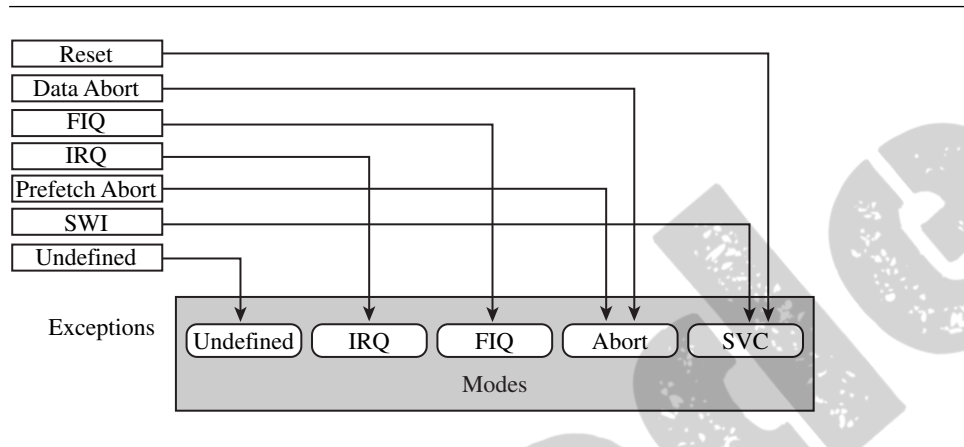


Figure 9.1 Exceptions and associated modes.

- sets the *cpsr* to the exception mode
- sets *pc* to the address of the exception handler

Figure 9.1 shows a simplified view of exceptions and associated modes. Note that when an exception occurs the ARM processor always switches to ARM state.

9.1.2 VECTOR TABLE

Chapter 2 introduced the *vector table*—a table of addresses that the ARM core branches to when an exception is raised. These addresses commonly contain branch instructions of one of the following forms:

- B <address>—This *branch instruction* provides a branch relative from the *pc*.
- LDR *pc*, [*pc*, #offset]—This *load register instruction* loads the handler address from memory to the *pc*. The address is an absolute 32-bit value stored close to the vector table. Loading this absolute literal value results in a slight delay in branching to a specific handler due to the extra memory access. However, you can branch to any address in memory.
- LDR *pc*, [*pc*, #-0xff0]—This *load register instruction* loads a specific interrupt service routine address from address 0xfffff030 to the *pc*. This specific instruction is only used when a vector interrupt controller is present (VIC PL190).

Table 9.2 Vector table and processor modes.

Exception	Mode	Vector table offset
Reset	SVC	+0x00
Undefined Instruction	UND	+0x04
Software Interrupt (SWI)	SVC	+0x08
Prefetch Abort	ABT	+0x0c
Data Abort	ABT	+0x10
Not assigned	—	+0x14
IRQ	IRQ	+0x18
FIQ	FIQ	+0x1c

- `MOV pc, #immediate`—This *move instruction* copies an immediate value into the *pc*. It lets you span the full address space but at limited alignment. The address must be an 8-bit immediate rotated right by an even number of bits.

You can also have other types of instructions in the vector table. For example, the FIQ handler might start at address offset +0x1c. Thus, the FIQ handler can start immediately at the FIQ vector location, since it is at the end of the vector table. The branch instructions cause the *pc* to jump to a specific location that can handle the specific exception.

Table 9.2 shows the exception, mode, and vector table offset for each exception.

EXAMPLE 9.1 Figure 9.2 shows a typical vector table. The Undefined Instruction entry is a branch instruction to jump to the undefined handler. The other vectors use an indirect address jump with the LDR load to *pc* instruction.

Notice that the FIQ handler also uses the LDR load to *pc* instruction and does not take advantage of the fact that the handler can be placed at the FIQ vector entry location. ■

```

0x00000000: 0xe59ffa38 RESET: > ldr pc, [pc, #reset]
0x00000004: 0xea000502 UNDEF: b undInstr
0x00000008: 0xe59ffa38 SWI : ldr pc, [pc, #swi]
0x0000000c: 0xe59ffa38 PABT : ldr pc, [pc, #prefetch]
0x00000010: 0xe59ffa38 DABT : ldr pc, [pc, #data]
0x00000014: 0xe59ffa38 - : ldr pc, [pc, #notassigned]
0x00000018: 0xe59ffa38 IRQ : ldr pc, [pc, #irq]
0x0000001c: 0xe59ffa38 FIQ : ldr pc, [pc, #fiq]
```

Figure 9.2 Example vector table.

9.1.3 EXCEPTION PRIORITIES

Exceptions can occur simultaneously, so the processor has to adopt a priority mechanism. Table 9.3 shows the various exceptions that occur on the ARM processor and their associated priority level. For instance, the Reset exception is the highest priority and occurs when power is applied to the processor. Thus, when a reset occurs, it takes precedence over all other exceptions. Similarly, when a Data Abort occurs, it takes precedence over all other exceptions apart from a Reset exception. The lowest priority level is shared by two exceptions, the Software Interrupt and Undefined Instruction exceptions. Certain exceptions also disable interrupts by setting the *I* or *F* bits in the *cpsr*, as shown in Table 9.3.

Each exception is dealt with according to the priority level set out in Table 9.3. The following is a summary of the exceptions and how they should be handled, starting with the highest.

The Reset exception is the highest priority exception and is always taken whenever it is signaled. The reset handler initializes the system, including setting up memory and caches. External interrupt sources should be initialized before enabling IRQ or FIQ interrupts to avoid the possibility of spurious interrupts occurring before the appropriate handler has been set up. The reset handler must also set up the stack pointers for all processor modes.

During the first few instructions of the handler, it is assumed that no exceptions or interrupts will occur. The code should be designed to avoid SWIs, undefined instructions, and memory accesses that may abort, that is, the handler is carefully implemented to avoid further triggering of an exception.

Data Abort exceptions occur when the memory controller or MMU indicates that an invalid memory address has been accessed (for example, if there is no physical memory for an address) or when the current code attempts to read or write to memory without the correct access permissions. An FIQ exception can be raised within a Data Abort handler since FIQ exceptions are not disabled. When the FIQ is completely serviced, control is returned back to the Data Abort handler.

A Fast Interrupt Request (FIQ) exception occurs when an external peripheral sets the FIQ pin to *nFIQ*. An FIQ exception is the highest priority interrupt. The core disables

Table 9.3 Exception priority levels.

Exceptions	Priority	<i>I</i> bit	<i>F</i> bit
Reset	1	1	1
Data Abort	2	1	—
Fast Interrupt Request	3	1	1
Interrupt Request	4	1	—
Prefetch Abort	5	1	—
Software Interrupt	6	1	—
Undefined Instruction	6	1	—

both IRQ and FIQ exceptions on entry into the FIQ handler. Thus, no external source can interrupt the processor unless the IRQ and/or FIQ exceptions are reenabled by software. It is desirable that the FIQ handler (and also the abort, SWI, and IRQ handlers) is carefully designed to service the exception efficiently.

An Interrupt Request (IRQ) exception occurs when an external peripheral sets the IRQ pin to $nIRQ$. An IRQ exception is the second-highest priority interrupt. The IRQ handler will be entered if neither an FIQ exception nor Data Abort exception occurs. On entry to the IRQ handler, the IRQ exceptions are disabled and should remain disabled until the current interrupt source has been cleared.

A Prefetch Abort exception occurs when an attempt to fetch an instruction results in a memory fault. This exception is raised when the instruction is in the execute stage of the pipeline and if none of the higher exceptions have been raised. On entry to the handler, IRQ exceptions will be disabled, but the FIQ exceptions will remain unchanged. If FIQ is enabled and an FIQ exception occurs, it can be taken while servicing the Prefetch Abort.

A Software Interrupt (SWI) exception occurs when the SWI instruction is executed and none of the other higher-priority exceptions have been flagged. On entry to the handler, the *cpsr* will be set to *supervisor* mode.

If the system uses nested SWI calls, the link register *r14* and *spsr* must be stored away before branching to the nested SWI to avoid possible corruption of the link register and the *spsr*.

An Undefined Instruction exception occurs when an instruction not in the ARM or Thumb instruction set reaches the execute stage of the pipeline and none of the other exceptions have been flagged. The ARM processor “asks” the coprocessors if they can handle this as a coprocessor instruction. Since coprocessors follow the pipeline, instruction identification can take place in the execute stage of the core. If none of the coprocessors claims the instruction, an Undefined Instruction exception is raised.

Both the SWI instruction and Undefined Instruction have the same level of priority, since they cannot occur at the same time (in other words, the instruction being executed cannot both be an SWI instruction and an undefined instruction).

9.1.4 LINK REGISTER OFFSETS

When an exception occurs, the link register is set to a specific address based on the current *pc*. For instance, when an IRQ exception is raised, the link register *lr* points to the last executed instruction plus 8. Care has to be taken to make sure the exception handler does not corrupt *lr* because *lr* is used to return from an exception handler. The IRQ exception is taken only after the current instruction is executed, so the return address has to point to the next instruction, or *lr* + 4. Table 9.4 provides a list of useful addresses for the different exceptions.

The next three examples show different methods of returning from an IRQ or FIQ exception handler.

Table 9.4 Useful link-register-based addresses.

Exception	Address	Use
Reset	—	<i>lr</i> is not defined on a Reset
Data Abort	<i>lr</i> - 8	points to the instruction that caused the Data Abort exception
FIQ	<i>lr</i> - 4	return address from the FIQ handler
IRQ	<i>lr</i> - 4	return address from the IRQ handler
Prefetch Abort	<i>lr</i> - 4	points to the instruction that caused the Prefetch Abort exception
SWI	<i>lr</i>	points to the next instruction after the SWI instruction
Undefined Instruction	<i>lr</i>	points to the next instruction after the undefined instruction

EXAMPLE 9.2 This example shows that a typical method of returning from an IRQ and FIQ handler is to use a SUBS instruction:

```

handler
    <handler code>
    ...
    SUBS    pc, r14, #4                ; pc=r14-4

```

Because there is an S at the end of the SUB instruction and the *pc* is the destination register, the *cpsr* is automatically restored from the *spsr* register. ■

EXAMPLE 9.3 This example shows another method that subtracts the offset from the link register *r14* at the beginning of the handler.

```

handler
    SUB     r14, r14, #4                ; r14-=4
    ...
    <handler code>
    ...
    MOVS    pc, r14                    ; return

```

After servicing is complete, return to normal execution occurs by moving the link register *r14* into the *pc* and restoring *cpsr* from the *spsr*. ■

EXAMPLE 9.4 The final example uses the interrupt stack to store the link register. This method first subtracts an offset from the link register and then stores it onto the interrupt stack.

```

handler
    SUB     r14, r14, #4                ; r14-=4

```

```

STMFD    r13!,{r0-r3, r14}          ; store context
...
<handler code>
...
LDMFD    r13!,{r0-r3, pc}^          ; return

```

To return to normal execution, the LDM instruction is used to load the *pc*. The ^ symbol in the instruction forces the *cpsr* to be restored from the *spsr*.

9.2 INTERRUPTS

There are two types of interrupts available on the ARM processor. The first type of interrupt causes an exception raised by an external peripheral—namely, IRQ and FIQ. The second type is a specific instruction that causes an exception—the SWI instruction. Both types suspend the normal flow of a program.

In this section we will focus mainly on IRQ and FIQ interrupts. We will cover these topics:

- Assigning interrupts
- Interrupt latency
- IRQ and FIQ exceptions
- Basic interrupt stack design and implementation

9.2.1 ASSIGNING INTERRUPTS

A system designer can decide which hardware peripheral can produce which interrupt request. This decision can be implemented in hardware or software (or both) and depends upon the embedded system being used.

An *interrupt controller* connects multiple external interrupts to one of the two ARM interrupt requests. Sophisticated controllers can be programmed to allow an external interrupt source to cause either an IRQ or FIQ exception.

When it comes to assigning interrupts, system designers have adopted a standard design practice:

- Software Interrupts are normally reserved to call privileged operating system routines. For example, an SWI instruction can be used to change a program running in *user* mode to a privileged mode. For an SWI handler example, take a look at Chapter 11.
- Interrupt Requests are normally assigned for general-purpose interrupts. For example, a periodic timer interrupt to force a context switch tends to be an IRQ exception. The IRQ exception has a lower priority and higher interrupt latency (to be discussed in the next section) than the FIQ exception.

- Fast Interrupt Requests are normally reserved for a single interrupt source that requires a fast response time—for example, direct memory access specifically used to move blocks of memory. Thus, in an embedded operating system design, the FIQ exception is used for a specific application, leaving the IRQ exception for more general operating system activities.

9.2.2 INTERRUPT LATENCY

Interrupt-driven embedded systems have to fight a battle with *interrupt latency*—the interval of time from an external interrupt request signal being raised to the first fetch of an instruction of a specific interrupt service routine (ISR).

Interrupt latency depends on a combination of hardware and software. System architects must balance the system design to handle multiple simultaneous interrupt sources and minimize interrupt latency. If the interrupts are not handled in a timely manner, then the system will exhibit slow response times.

Software handlers have two main methods to minimize interrupt latency. The first method is to use a *nested interrupt handler*, which allows further interrupts to occur even when currently servicing an existing interrupt (see Figure 9.3). This is achieved by reenabling the interrupts as soon as the interrupt source has been serviced (so it won't generate more interrupts) but before the interrupt handling is complete. Once a nested interrupt has been serviced, then control is relinquished to the original interrupt service routine.

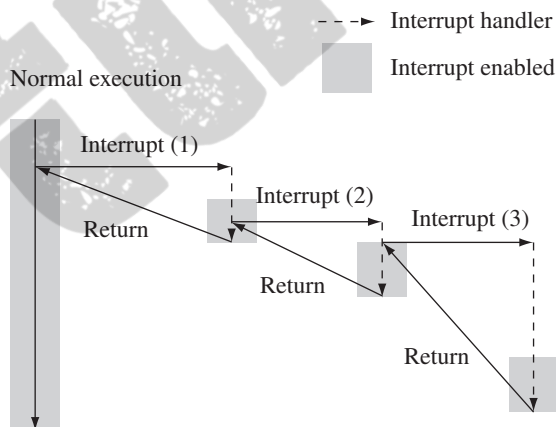


Figure 9.3 A three-level nested interrupt.

The second method involves *prioritization*. You program the interrupt controller to ignore interrupts of the same or lower priority than the interrupt you are handling, so only a higher-priority task can interrupt your handler. You then reenables interrupts.

The processor spends time in the lower-priority interrupts until a higher-priority interrupt occurs. Therefore higher-priority interrupts have a lower average interrupt latency than the lower-priority interrupts, which reduces latency by speeding up the completion time on the critical time-sensitive interrupts.

9.2.3 IRQ AND FIQ EXCEPTIONS

IRQ and FIQ exceptions only occur when a specific interrupt mask is cleared in the *cpsr*. The ARM processor will continue executing the current instruction in the execution stage of the pipeline before handling the interrupt—an important factor in designing a deterministic interrupt handler since some instructions require more cycles to complete the execution stage.

An IRQ or FIQ exception causes the processor hardware to go through a standard procedure (provided the interrupts are not masked):

1. The processor changes to a specific interrupt request mode, which reflects the interrupt being raised.
2. The previous mode's *cpsr* is saved into the *spsr* of the new interrupt request mode.
3. The *pc* is saved in the *lr* of the new interrupt request mode.
4. *Interrupt/s are disabled*—either the IRQ or both IRQ and FIQ exceptions are disabled in the *cpsr*. This immediately stops another interrupt request of the same type being raised.
5. The processor branches to a specific entry in the vector table.

The procedure varies slightly depending upon the type of interrupt being raised. We will illustrate both interrupts with an example. The first example shows what happens when an IRQ exception is raised, and the second example shows what happens when an FIQ exception is raised.

EXAMPLE 9.5 Figure 9.4 shows what happens when an IRQ exception is raised when the processor is in *user* mode. The processor starts in state 1. In this example both the IRQ and FIQ exception bits in the *cpsr* are enabled.

When an IRQ occurs the processor moves into state 2. This transition automatically sets the IRQ bit to one, disabling any further IRQ exceptions. The FIQ exception, however, remains enabled because FIQ has a higher priority and therefore does not get disabled when a low-priority IRQ exception is raised. The *cpsr* processor mode changes to *IRQ* mode. The *user* mode *cpsr* is automatically copied into *spsr_irq*.

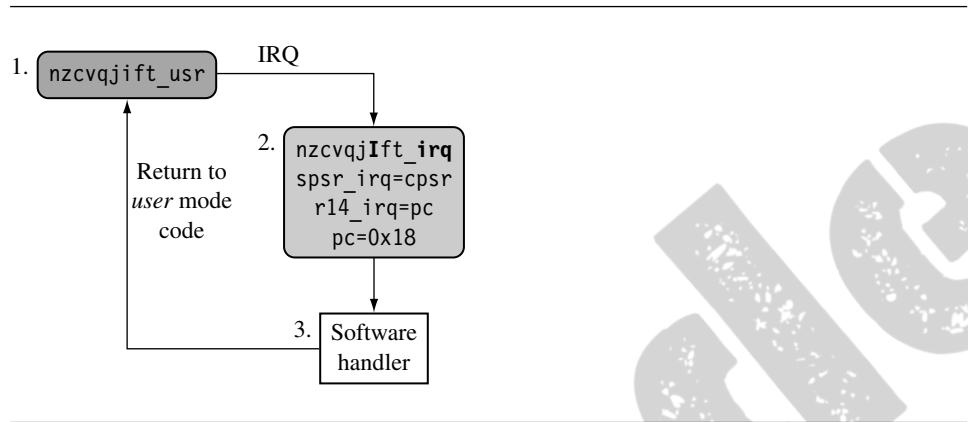


Figure 9.4 Interrupt Request (IRQ).

Register *r14_irq* is assigned the value of the *pc* when the interrupt was raised. The *pc* is then set to the IRQ entry +0x18 in the vector table.

In state 3 the software handler takes over and calls the appropriate interrupt service routine to service the source of the interrupt. Upon completion, the processor mode reverts back to the original *user* mode code in state 1.

EXAMPLE 9.6 Figure 9.5 shows an example of an FIQ exception. The processor goes through a similar procedure as with the IRQ exception, but instead of just masking further IRQ exceptions from occurring, the processor also masks out further FIQ exceptions. This means that both interrupts are disabled when entering the software handler in state 3.

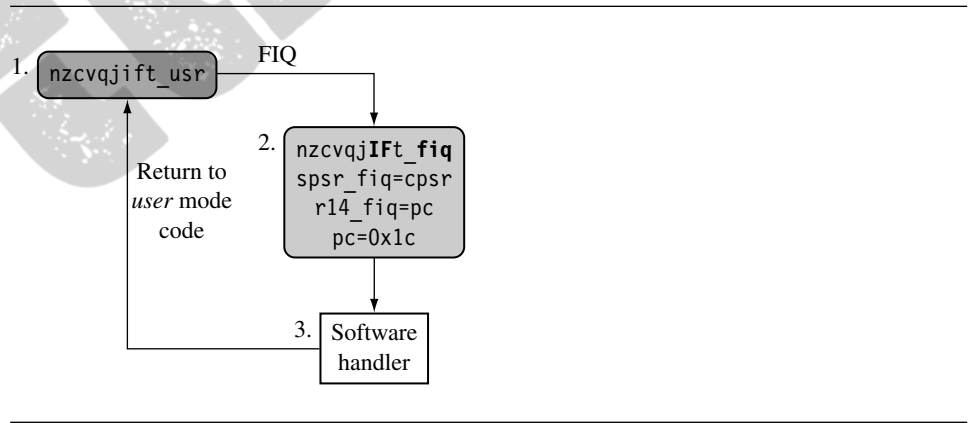


Figure 9.5 Fast Interrupt Request (FIQ).

Changing to *FIQ* mode means there is no requirement to save registers *r8* to *r12* since these registers are banked in *FIQ* mode. These registers can be used to hold temporary data, such as buffer pointers or counters. This makes *FIQ* ideal for servicing a single-source, high-priority, low-latency interrupt.

9.2.3.1 Enabling and Disabling FIQ and IRQ Exceptions

The ARM processor core has a simple procedure to manually enable and disable interrupts that involves modifying the *cpsr* when the processor is in a privileged mode.

Table 9.5 shows how IRQ and FIQ interrupts are enabled. The procedure uses three ARM instructions.

The first instruction *MRS* copies the contents of the *cpsr* into register *r1*. The second instruction clears the IRQ or FIQ mask bit. The third instruction then copies the updated contents in register *r1* back into the *cpsr*, enabling the interrupt request. The postfix *_c* identifies that the bit field being updated is the control field bit [7:0] of the *cpsr*. (For more details see Chapter 2.) Table 9.6 shows a similar procedure to disable or mask an interrupt request.

It is important to understand that the interrupt request is either enabled or disabled only once the *MSR* instruction has completed the execution stage of the pipeline. Interrupts can still be raised or masked prior to the *MSR* completing this stage.

Table 9.5 Enabling an interrupt.

<i>cpsr</i> value	IRQ	FIQ
Pre	<i>nzcvqjIFt_SVC</i>	<i>nzcvqjIFt_SVC</i>
Code	<i>enable_irq</i>	<i>enable_fiq</i>
	<i>MRS</i> <i>r1</i> , <i>cpsr</i>	<i>MRS</i> <i>r1</i> , <i>cpsr</i>
	<i>BIC</i> <i>r1</i> , <i>r1</i> , #0x80	<i>BIC</i> <i>r1</i> , <i>r1</i> , #0x40
	<i>MSR</i> <i>cpsr_c</i> , <i>r1</i>	<i>MSR</i> <i>cpsr_c</i> , <i>r1</i>
Post	<i>nzcvqjiFt_SVC</i>	<i>nzcvqjiFt_SVC</i>

Table 9.6 Disabling an interrupt.

<i>cpsr</i>	IRQ	FIQ
Pre	<i>nzcvqjiFt_SVC</i>	<i>nzcvqjiFt_SVC</i>
Code	<i>disable_irq</i>	<i>disable_fiq</i>
	<i>MRS</i> <i>r1</i> , <i>cpsr</i>	<i>MRS</i> <i>r1</i> , <i>cpsr</i>
	<i>ORR</i> <i>r1</i> , <i>r1</i> , #0x80	<i>ORR</i> <i>r1</i> , <i>r1</i> , #0x40
	<i>MSR</i> <i>cpsr_c</i> , <i>r1</i>	<i>MSR</i> <i>cpsr_c</i> , <i>r1</i>
Post	<i>nzcvqjIFt_SVC</i>	<i>nzcvqjIFt_SVC</i>

To enable and disable both the IRQ and FIQ exceptions requires a slight modification to the second instruction. The immediate value on the data processing BIC or ORR instruction has to be changed to 0xc0 to enable or disable both interrupts.

9.2.4 BASIC INTERRUPT STACK DESIGN AND IMPLEMENTATION

Exceptions handlers make extensive use of stacks, with each mode having a dedicated register containing the stack pointer. The design of the exception stacks depends upon these factors:

- *Operating system requirements*—Each operating system has its own requirements for stack design.
- *Target hardware*—The target hardware provides a physical limit to the size and positioning of the stack in memory.

Two design decisions need to be made for the stacks:

- The *location* determines where in the memory map the stack begins. Most ARM-based systems are designed with a stack that descends downwards, with the top of the stack at a high memory address.
- *Stack size* depends upon the type of handler, nested or nonnested. A nested interrupt handler requires more memory space since the stack will grow with the number of nested interrupts.

A good stack design tries to avoid *stack overflow*—where the stack extends beyond the allocated memory—because it causes instability in embedded systems. There are software techniques that identify overflow and that allow corrective measures to take place to repair the stack before irreparable memory corruption occurs. The two main methods are (1) to use memory protection and (2) to call a stack check function at the start of each routine.

The IRQ mode stack has to be set up before interrupts are enabled—normally in the initialization code for the system. It is important that the maximum size of the stack is known in a simple embedded system, since the stack size is reserved in the initial stages of boot-up by the firmware.

Figure 9.6 shows two typical memory layouts in a linear address space. The first layout, *A*, shows a traditional stack layout with the interrupt stack stored underneath the code segment. The second layout, *B*, shows the interrupt stack at the top of the memory above the user stack. The main advantage of layout *B* over *A* is that *B* does not corrupt the vector table when a stack overflow occurs, and so the system has a chance to correct itself when an overflow has been identified.

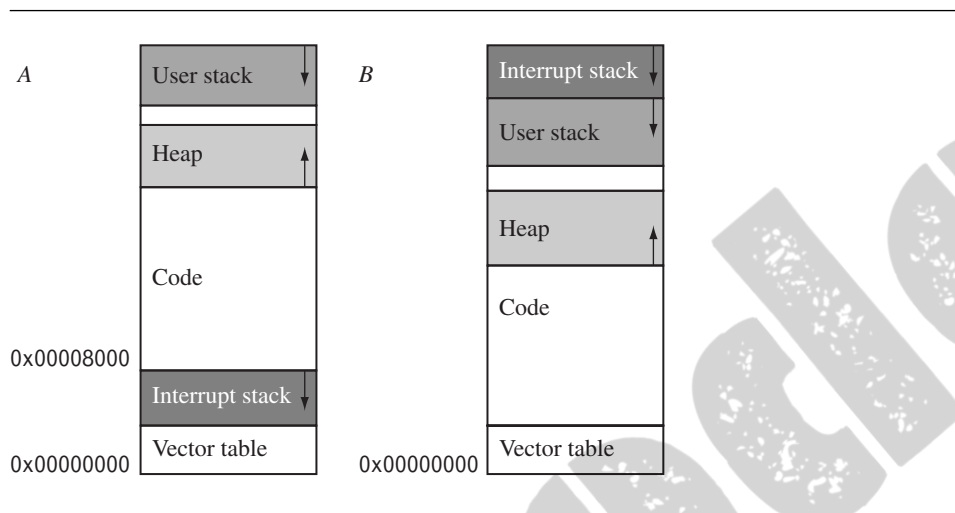


Figure 9.6 Typical memory layouts.

EXAMPLE 9.7 For each processor mode a stack has to be set up. This is carried out every time the processor is reset. Figure 9.7 shows an implementation using layout A. To help set up the memory layout, a set of defines are declared that map the memory region names with an absolute address.

For instance, the User stack is given the label `USR_Stack` and is set to address `0x20000`. The Supervisor stack is set to an address that is 128 bytes below the IRQ stack.

```
USR_Stack    EQU 0x20000
IRQ_Stack    EQU 0x8000
SVC_Stack    EQU IRQ_Stack-128
```

To help change to the different processor modes, we declare a set of defines that map each processor mode with a particular mode bit pattern. These labels can then be used to set the *cpsr* to a new mode.

```
Usr32md      EQU 0x10           ; User mode
FIQ32md      EQU 0x11           ; FIQ mode
IRQ32md      EQU 0x12           ; IRQ mode
SVC32md      EQU 0x13           ; Supervisor mode
Abt32md      EQU 0x17           ; Abort mode
Und32md      EQU 0x1b           ; Undefined instruction mode
Sys32md      EQU 0x1f           ; System mode
```

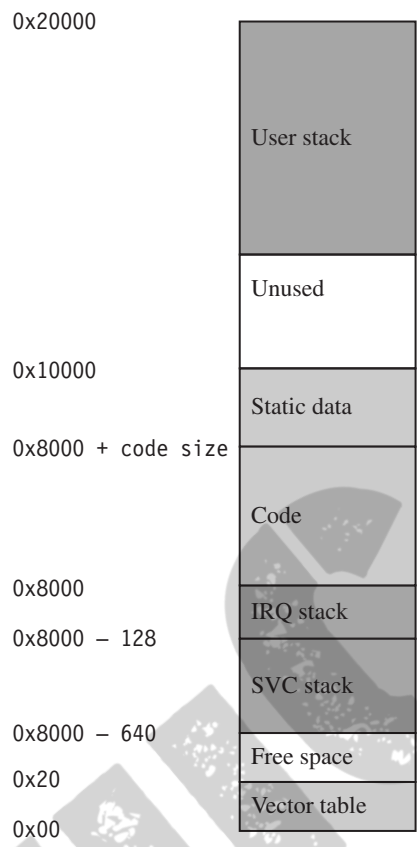


Figure 9.7 Example implementation using layout A.

For safety reasons a define is declared to disable both the IRQ and FIQ exceptions in the *cpsr*:

```
NoInt          EQU 0xc0          ; Disable interrupts
```

NoInt masks both interrupts by setting the masks to one.

Initialization code starts by setting up the stack registers for each processor mode. The stack register *r13* is one of the registers that is always banked when a mode change occurs. The code first initializes the IRQ stack. For safety reasons, it is always best to make sure that interrupts are disabled by using a bitwise OR between NoInt and the new mode.

Each mode stack must be set up. Here is an example of how to set up three different stacks when the processor core comes out of *reset*. Note that, since this is a basic example, we do not implement a stack for the *abort*, *FIQ*, and *undefined instruction* modes. If these stacks are required, then very similar code is used.

- *Supervisor mode stack*—The processor core starts in *supervisor* mode so the SVC stack setup involves loading register *r13_svc* with the address pointed to by *SVC_NewStack*. For this example the value is *SVC_Stack*.

```

        LDR    r13, SVC_NewStack      ; r13_svc
        ...
SVC_NewStack
        DCD    SVC_Stack

```

- *IRQ mode stack*—To set up the IRQ stack, the processor mode has to change to *IRQ* mode. This is achieved by storing a *cpsr* bit pattern into register *r2*. Register *r2* is then copied into the *cpsr*, placing the processor into *IRQ* mode. This action immediately makes register *r13_irq* viewable, and it can then be assigned the *IRQ_Stack* value.

```

        MOV    r2, #NoInt|IRQ32md
        MSR    cpsr_c, r2
        LDR    r13, IRQ_NewStack     ; r13_irq
        ...
IRQ_NewStack
        DCD    IRQ_Stack

```

- *User mode stack*—It is common for the *user* mode stack to be the last to be set up because when the processor is in *user* mode there is no direct method to modify the *cpsr*. An alternative is to force the processor into *system* mode to set up the *user* mode stack since both modes share the same registers.

```

        MOV    r2, #Sys32md
        MSR    cpsr_c, r2
        LDR    r13, USR_NewStack     ; r13_usr
        ...
USR_NewStack
        DCD    USR_Stack

```

Using separate stacks for each mode rather than processing using a single stack has one main advantage: errant tasks can be debugged and isolated from the rest of the system. ■

CHAPTER 10

FIRMWARE

This chapter discusses firmware for ARM-based embedded systems. Firmware is an important part of any embedded system since it is frequently the first code to be ported and executed on a new platform. Firmware can vary from being a complete software embedded system to just a simple initialization and bootloader routine. We have divided this chapter into two sections.

The first section introduces firmware. In this section we define the term *firmware* and describe two popular industry standard firmware packages available for the ARM processor—ARM Firmware Suite and Red Hat’s RedBoot. These firmware packages are general purpose and can be ported to different ARM platforms relatively easily and quickly.

The second section focuses on just the initialization and bootloader process. To help with this, we have developed a simple example called Sandstone. Sandstone is designed to initialize hardware, load an image into memory, and relinquish control of the *pc* over to that image.

We start by first discussing firmware and introduce the two common ARM firmware packages.

10.1 FIRMWARE AND BOOTLOADER

We realize that the use of terms may differ among engineers, but we will use the following definitions:

- The *firmware* is the deeply embedded, low-level software that provides an interface between the hardware and the application/operating system level software. It resides in the ROM and executes when power is applied to the embedded hardware system. Firmware can remain active after system initialization and supports basic

system operations. The choice of which firmware to use for a particular ARM-based system depends upon the specific application, which can range from loading and executing a sophisticated operating system to simply relinquishing control to a small microkernel. Consequently, requirements can vary greatly from one firmware implementation to another. For example, a small system may require just minimal firmware support to boot a small operating system. One of the main purposes of firmware is to provide a stable mechanism to load and boot an operating system.

- The *bootloader* is a small application that installs the operating system or application onto a hardware target. The bootloader only exists up to the point that the operating system or application is executing, and it is commonly incorporated into the firmware.

To help understand the features of different firmware implementations, we have a common execution flow (see Table 10.1). Each stage is now discussed in more detail.

The first stage is to set up the target platform—in other words, prepare the environment to boot an operating system since an operating system expects a particular type of environment before it can operate. This step involves making sure that the platform is correctly initialized (for example, making sure that the control registers of a particular microcontroller are placed at a known address or changing the memory map to an expected layout).

It is common for the same executable to operate on different cores and platforms. In this case, the firmware has to identify and discover the exact core and platform it is operating on. The core is normally recognized by reading register 0 in coprocessor 15, which holds both the processor type and the manufacturer name. There are multiple ways to identify the platform, from checking for the existence of a set of particular peripherals to simply reading a preprogrammed chip.

Table 10.1 Firmware execution flow.

Stage	Features
Set up target platform	Program the hardware system registers Platform identification Diagnostics Debug interface Command line interpreter
Abstract the hardware	Hardware Abstraction Layer Device driver
Load a bootable image	Basic filing system
Relinquish control	Alter the <i>pc</i> to point into the new image

Diagnostics software provides a useful way for quickly identifying basic hardware malfunctions. Because of the nature of this type of software, it tends to be specific to a particular piece of hardware.

Debug capability is provided in the form of a module or monitor that provides software assistance for debugging code running on a hardware target. This assistance includes the following:

- Setting up breakpoints in RAM. A breakpoint allows a program to be interrupted and the state of the processor core to be examined.
- Listing and modifying memory (using peek and poke operations).
- Showing current processor register contents.
- Disassembling memory into ARM and Thumb instruction mnemonics.

These are interactive functions: you can either send the commands through a command line interpreter (CLI) or through a dedicated host debugger attached to the target platform. Unless the firmware has access to the internal hardware debug circuitry, only RAM images can be debugged through a software debug mechanism.

The CLI is commonly available on the more advanced firmware implementations. It allows you to change the operating system to be booted by altering the default configurations through typing commands at a command prompt. For embedded systems, the CLI is commonly controlled through a host terminal application. Communication between the host and the target is normally over a serial or network connection.

The second stage is to abstract the hardware. The *Hardware Abstraction Layer* (HAL) is a software layer that hides the underlying hardware by providing a set of defined programming interfaces. When you move to a new target platform, these programming interfaces remain constant but the underlying implementation changes. For instance, two target platforms might use a different timer peripheral. Each peripheral would require new code to initialize and configure the device. The HAL programming interface would remain unaltered even though both the hardware and software may differ greatly between implementations.

The HAL software that communicates with specific hardware peripherals is called a *device driver*. A device driver provides a standard application programming interface (API) to read and write to a specific peripheral.

The third stage is to load a bootable image. The ability of firmware to carry out this activity depends upon the type of media used to store the image. Note that not all operating system images or application images need to be copied into RAM. The operating system image or application image can simply execute directly from ROM.

ARM processors are normally found in small devices that include flash ROM. A common feature is a simple flash ROM filing system (FFS), which allows multiple executable images to be stored.

Other media devices, such as hard drives, require that the firmware incorporates a device driver that is suitable for accessing the hardware. Accessing the hardware requires

that the firmware has knowledge of the underlying filing system format, which gives the firmware the ability to read the filing system, find the file that contains the image, and copy the image into memory. Similarly, if the image is on the network, then the firmware must also understand the network protocol as well as the Ethernet hardware.

The load process has to take into account the image format. The most basic image format is plain binary. A plain binary image does not contain any header or debug information. A popular image format for ARM-based systems is Executable and Linking Format (ELF). This format was originally developed for UNIX systems and replaced the older format called Common Object File Format (COFF). ELF files come in three forms: relocatable, executable, and shared object.

Most firmware systems must deal with the executable form. Loading an ELF image involves deciphering the standard ELF header information (that is, execution address, type, size, and so on). The image may also be encrypted or compressed, in which case the load process would involve performing decryption or decompression on the image.

The fourth stage is to relinquish control. This is where the firmware hands over control of the platform to an operating system or application. Note that not all firmware hands over control; instead the firmware can remain the controlling software on the platform.

Firmware designed to pass control to an operating system may become inactive once the operating system has control. Alternatively, the Machine Independent Layer (MIL) or Hardware Abstraction Layer (HAL) part of the firmware can remain active. This layer exposes, through the SWI mechanism, a standard application interface for specific hardware devices.

Relinquishing control on an ARM system means updating the vector table and modifying the *pc*. Updating the vector table involves modifying particular exception and interrupt vectors so that they point to specialized operating system handlers. The *pc* has to be modified so that it points to the operating system entry point address.

For more sophisticated operating systems, such as Linux, relinquishing control requires that a standard data structure be passed to the kernel. This data structure explains the environment that the kernel will be running in. For example, one field may include the amount of available RAM on the platform, while another field includes the type of MMU being used.

We use these definitions to describe two common firmware suites.

10.1.1 ARM FIRMWARE SUITE

ARM has developed a firmware package called the ARM Firmware Suite (AFS). AFS is designed purely for ARM-based embedded systems. It provides support for a number of boards and processors including the Intel XScale and StrongARM processors. The package includes two major pieces of technology, a Hardware Abstraction Layer called μ HAL (pronounced micro-HAL) and a debug monitor called Angel.

μ HAL provides a low-level device driver framework that allows it to operate over different communication devices (for example, USB, Ethernet, or serial). It also provides a

standard API. Consequently, when a port takes place, the various hardware-specific parts must be implemented in accordance with the various μ HAL API functions.

This has the advantage of making the porting process relatively straightforward since you have a standard function framework to work within. Once the firmware is ported, the task of moving an operating system over to the new target platform can take place. The speed of this activity depends upon whether the OS takes advantage of the ported μ HAL API call to access the hardware.

μ HAL supports these main features:

- *System initialization*—setting up the target platform and processor core. Depending upon the complexity of the target platform, this can either be a simple or complicated task.
- *Polled serial driver*—used to provide a basic method of communication with a host.
- *LED support*—allows control over the LEDs for simple user feedback. This provides an application the ability to display operational status.
- *Timer support*—allows a periodic interrupt to be set up. This is essential for preemptive context switching operating systems that require this mechanism.
- *Interrupt controllers*—support for different interrupt controllers.

The boot monitor in μ HAL contains a CLI.

The second technology, Angel, allows communication between a host debugger and a target platform. It allows you to inspect and modify memory, download and execute images, set breakpoints, and display processor register contents. All this control is through the host debugger. The Angel debug monitor must have access to the SWI and IRQ or FIQ vectors.

Angel uses SWI instructions to provide a set of APIs that allow a program to open, read, and write to a host filing system. IRQ/FIQ interrupts are used for communication purposes with the host debugger.

10.1.2 RED HAT REDBOOT

RedBoot is a firmware tool developed by Red Hat. It is provided under an open source license with no royalties or up front fees. RedBoot is designed to execute on different CPUs (for instance, ARM, MIPS, SH, and so on). It provides both debug capability through GNU Debugger (GDB), as well as a bootloader. The RedBoot software core is based on a HAL.

RedBoot supports these main features:

- *Communication*—configuration is over serial or Ethernet. For serial, X-Modem protocol is used to communicate with the GNU Debugger (GDB). For Ethernet, TCP is used to communicate with GDB. RedBoot supports a range of network standards, such as *bootp*, *telnet*, and *tftp*.

- *Flash ROM memory management*—provides a set of filing system routines that can download, update, and erase images in flash ROM. In addition, the images can either be compressed or uncompressed.
- *Full operating system support*—supports the loading and booting of Embedded Linux, Red Hat eCos, and many other popular operating systems. For Embedded Linux, RedBoot supports the ability to define parameters that are passed directly to the kernel upon booting.

10.2 EXAMPLE: SANDSTONE

We have designed Sandstone to be a minimal system. It carries out only the following tasks: set up target platform environment, load a bootable image into memory, and relinquish control to an operating system. It is, however, still a real working example.

The implementation is specific to the ARM Evaluator-7T platform, which includes an ARM7TDMI processor. This example shows you exactly how a simple platform can be set up and a software payload can be loaded into memory and booted. The payload can either be an application or operating system image. Sandstone is a static design and cannot be configured after the build process is complete. Table 10.2 lists the basic characteristics of Sandstone.

We will walk you through the directory layout and code structure. The directory layout shows you where the source code is located and where the different build files are placed. The code structure focuses more on the actual initialization and boot process.

Note that Sandstone is written entirely in ARM assembler and is a working piece of code that can be used to initialize target hardware and boot any piece of software, within reason, on the ARM Evaluator-7T.

10.2.1 SANDSTONE DIRECTORY LAYOUT

Sandstone can be found on our Web site. If you take a look at Sandstone, you will see that the directory structure is as shown in Figure 10.1. The structure follows a standard style

Table 10.2 Summary of Sandstone.

Feature	Configuration
Code	ARM instructions only
Tool chain	ARM Developer Suite 1.2
Image size	700 bytes
Source	17 KB
Memory	remapped

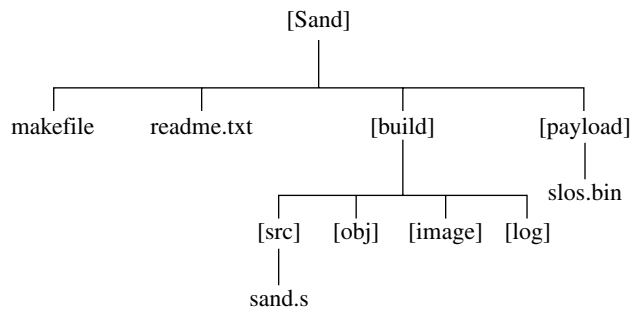


Figure 10.1 Sandstone directory layout.

that we will continue to use in further chapters. The sandstone source file `sand.s` is located under the `sand/build/src` directory.

The object file produced by the assembler is placed under the `build/obj` directory. The object file is then linked, and the final Sandstone image is placed under the `sand/build/image` directory. This image includes both the Sandstone code and the payload. The payload image, the image that is loaded and booted by Sandstone, is found under the `sand/payload` directory.

For information about the Sandstone build procedure, take a look at the `readme.txt` file under the `sand` directory. This file contains a description of how to build the example binary image for the ARM Evaluator-7T.

10.2.2 SANDSTONE CODE STRUCTURE

Sandstone consists of a single assembly file. The file structure is broken down into a number of steps, where each step corresponds to a stage in the execution flow of Sandstone (see Table 10.3).

Table 10.3 Sandstone execution flow.

Step	Description
1	Take the Reset exception
2	Start initializing the hardware
3	Remap memory
4	Initialize communication hardware
5	Bootloader—copy payload and relinquish control

We will take you through these steps, trying to avoid as much as possible the platform-specific parts. You should note that some specific parts are unavoidable (for example, configuring system registers and memory remapping).

The initial goal of Sandstone is to set up the target platform environment so that it can provide some form of feedback to indicate that the firmware is running and has control of the platform.

10.2.2.1 Step 1: Take the Reset Exception

Execution begins with a Reset exception. Only the reset vector entry is required in the default vector table. It is the very first instruction executed. You can see from the code here that all the vectors, apart from the reset vector, branch to a unique *dummy handler*—a branch instruction that causes an infinite loop. It is assumed that no exception or interrupt will occur during the operation of Sandstone. The reset vector is used to move the execution flow to the second stage.

```

AREA start, CODE, READONLY
ENTRY

sandstone_start
    B    sandstone_init1    ; reset vector
    B    ex_und              ; undefined vector
    B    ex_swi              ; swi vector
    B    ex_pabt             ; prefetch abort vector
    B    ex_dabt             ; data abort vector
    NOP                    ; not used...
    B    int_irq             ; irq vector
    B    int_fiq             ; fiq vector

ex_und B    ex_und          ; loop forever
ex_swi B    ex_swi          ; loop forever
ex_dabt B    ex_dabt        ; loop forever
ex_pabt B    ex_pabt        ; loop forever
int_irq B    int_irq        ; loop forever
int_fiq B    int_fiq        ; loop forever

```

sandstone_start is located at address 0x00000000.

The results of executing step 1 are the following:

- Dummy handlers are set up.
- Control is passed to code to initialize the hardware.

10.2.2.2 Step 2: Start Initializing the Hardware

The primary phase in initializing hardware is setting up system registers. These registers have to be set up before accessing the hardware. For example, the ARM Evaluator-7T has a seven-segment display, which we have chosen to be used as a feedback tool to indicate that the firmware is active. Before we can set up the segment display, we have to position the base address of the system registers to a known location. In this case, we have picked the default address 0x03ff0000, since this places all the hardware system registers away from both ROM and RAM, separating the peripherals and memory.

Consequently, all the microcontroller memory-mapped registers are located as an offset from 0x03ff0000. This is achieved using the following code:

```
sandstone_init1
    LDR    r3, =SYSCFG      ; where SYSCFG=0x03ff0000
    LDR    r4, =0x03ffffa0
    STR    r4, [r3]
```

Register *r3* contains the default system register base address and is used to set the new default address, as well as other specific attributes such as the cache. Register *r4* contains the new configuration. The top 16 bits contain the high address of the new system register base address 0x03ff, and the lower 16 bits contain the new attribute settings 0xffa0.

After setting up the system register base address, the segment display can be configured. The segment display hardware is used to show Sandstone's progress. Note that the segment display is not shown since it is hardware specific.

The results of executing step 2 are the following:

- The system registers are set from a known base address—0x03ff0000.
- The segment display is configured, so that it can be used to display progress.

10.2.2.3 Step 3: Remap Memory

One of the major activities of hardware initialization is to set up the memory environment. Sandstone is designed to initialize SRAM and remap memory. This process occurs fairly early on in the initialization of the system. The platform starts in a known memory state, as shown in Table 10.4.

As you can see, when the platform is powered up, only flash ROM is assigned a location in the memory map. The two SRAM banks (0 and 1) have not been initialized and are not available. The next stage is to bring in the two SRAM banks and remap flash ROM to a new location. This is achieved using the following code:

```
LDR    r14, =sandstone_init2
LDR    r4, =0x01800000      ; new flash ROM location
```

Table 10.4 Initial memory state.

Memory type	Start address	End address	Size
Flash ROM	0x00000000	0x00080000	512K
SRAM bank 0	Unavailable	unavailable	256K
SRAM bank 1	Unavailable	unavailable	256K

```

ADD    r14, r14, r4
ADRL   r0, memorymaptable_str
LDMIA  r0, {r1-r12}
LDR     r0, =EXTDBWTH           ; =(SYSCFG + 0x3010)
STMIA  r0, {r1-r12}
MOV     pc, r14                 ; jump to remapped memory

```

```

sandstone_init2
; Code after sandstone_init2 executes @ +0x1800000

```

The first part of the code calculates the absolute address of the routine *sandstone_init2* before remapping takes place. This address is used by Sandstone to jump to the next routine in the new remapped environment.

The second part carries out the memory remapping. The new memory map data is loaded into registers *r1* to *r12*, from a structure pointed by *memorymaptable_str*. This structure, using the registers, is then written to the memory controller offset 0x3010 from system configuration register. Once this is complete, the new memory map as shown in Table 10.5 is active.

You can see that the SRAM banks are now available, and the flash ROM is set to a higher address. The final part is to jump to the next routine, or stage, of the firmware.

This jump is achieved by taking advantage of the ARM pipeline. Even though the new memory environment is active, the next instruction has already been loaded into the pipeline. The next routine can be called by moving the contents of register *r14* (the address *sandstone_init2*) into the *pc*. We achieve this by using a single MOV instruction that follows immediately after the remap code.

Table 10.5 Remapping.

Type	Start address	End address	Size
Flash ROM	0x01800000	0x01880000	512K
SRAM bank 0	0x00000000	0x00040000	256K
SRAM bank 1	0x00040000	0x00080000	256K

The results of executing step 3 are the following:

- Memory has been remapped as shown in Table 10.5.
- *pc* now points to the next step. This address is located in the newly remapped flash ROM.

10.2.2.4 Step 4: Initialize Communication Hardware

Communication initialization involves configuring a serial port and outputting a standard banner. The banner is used to show that the firmware is fully functional and memory has been successfully remapped. Again, because the code for initializing the serial port on the ARM Evaluator-7T is hardware specific, it is not shown.

The serial port is set to 9600 baud, no parity, one stop bit, and no flow control. If a serial cable is attached to the board, then the host terminal has to be configured with these settings.

The results of executing step 4 are the following:

- Serial port initialized—9600 baud, no parity, one stop bit, and no flow control.
- Sandstone banner sent out through the serial port:

```
Sandstone Firmware (0.01)
- platform ..... e7t
- status ..... alive
- memory ..... remapped
+ booting payload ...
```

10.2.2.5 Step 5: Bootloader—Copy Payload and Relinquish Control

The final stage involves copying a payload and relinquishing control of the *pc* over to the copied payload. This is achieved using the code shown here. The first part of the code sets up the registers *r12*, *r13*, and *r14* used in the block copy. The bootloader code assumes that the payload is a plain binary image that requires no deciphering or uncompressing.

```
sandstone_load_and_boot
    MOV     r13,#0                ; destination addr
    LDR     r12,payload_start_addr ; start addr
    LDR     r14,payload_end_addr   ; end addr
```

```

_copy
    LDMIA    r12!,{r0-r11}
    STMIA    r13!,{r0-r11}
    CMP      r12,r14
    BLT      _copy
    MOV      pc,#0

payload_start_address
    DCD      startAddress
payload_end_address
    DCD      endAddress

```

Destination register *r13* points to the beginning of SRAM, in this case 0x00000000. The source register *r12* points to the start of the payload, and the source end register *r14* points to the end of the payload. Using these registers, the payload is then copied into SRAM.

Control of the *pc* is then relinquished to the payload by forcing the *pc* to the entry address of the copied payload. For this particular payload the entry point is address 0x00000000. The payload now has control of the system.

The results of executing step 5 are the following:

- Payload copied into SRAM, address 0x00000000.
- Control of the *pc* is relinquished to the payload; *pc* = 0x00000000.
- The system is completely booted. The following output is shown on the serial port:

```

Sandstone Firmware (0.01)
- platform ..... e7t
- status ..... alive
- memory ..... remapped

+ booting payload ...

Simple Little OS (0.09)
- initialized ..... ok
- running on ..... e7t

e7t:

```

10.3 SUMMARY

This chapter covered the firmware. We defined *firmware* as the low-level code that interfaces the hardware with an application or operating system. We also defined the *bootloader* as the software that loads an operating system or application into memory and relinquishes control of the *pc* to that software.

We introduced the ARM Firmware Suite and RedBoot. The ARM Firmware Suite is designed only for ARM-based systems. RedBoot, however, is more generic and can be used on other, non-ARM processors.

Next we looked at a firmware example called Sandstone. Sandstone initializes the hardware and then loads and boots an image following this procedure:

- Takes the Reset exception.
- Starts initializing the hardware; sets the system register's base address and initializes segment display hardware.
- Remaps memory; ROM address = high addr and SRAM addr = 0x00000000.
- Initializes the communication hardware output on the serial port.
- Bootloader—loads an image into SRAM and relinquishes control of the *pc* to the image (*pc* = 0x00000000).

We now have a fully initialized ARM7TDMI embedded system.